

1. Структурное vs Процедурное программирование

Структурное программирование — это методология, основанная на использовании трех управляющих конструкций: последовательности, ветвления (if-else) и циклов (for, while).

Процедурное программирование — это способ организации кода, при котором программа разбивается на процедуры (функции) для переиспользования кода.

Отличие: Структурное программирование — это «фундамент» (как мы пишем внутри функций), а процедурное — это архитектура (как мы группируем код).

2. Основные концепции ООП

Смысл ООП — отображение реальных объектов и их взаимодействий в коде.

Инкапсуляция: объединение данных и методов работы с ними в одном классе и сокрытие деталей.

Наследование: возможность создания новых классов на основе существующих. Смысл: переиспользование кода.

Полиморфизм: способность объектов с одинаковой спецификацией (интерфейсом) иметь разную реализацию.

Абстракция: выделение существенных характеристик объекта и игнорирование второстепенных.

3. Основные понятия ООП

Класс или «шаблон».

Объект — это конкретный экземпляр класса, занимающий место в памяти.

Атрибут — это переменная (свойство), описывающая характеристику или состояние объекта.

Задача

А) Создание простого класса и объекта: создать класс `Person` с атрибутами `name`, `age` и методами `getName()`, `setName(name)`; реализовать метод `displayInfo()`, который выводит информацию об объекте; создать экземпляр класса `Person` и продемонстрировать работу методов. Б) Использование наследования: создать новый класс `Student`, который наследует от класса `Person`. Добавьте атрибут `studentId` и метод `getStudentId()`; переопределите метод `displayInfo()` в классе `Student`, чтобы он выводил дополнительную информацию; создайте экземпляр класса `Student` и продемонстрируйте работу унаследованных и переопределенных методов. В) Применение полиморфизма: определите абстрактный класс `Shape` с методом `calculateArea()`; создайте два конкретных класса-наследника: `Rectangle` и `Circle`, реализующие метод `calculateArea()` соответствующим образом; создайте массив объектов разных форм (`Shape[]`) и выполните

итерацию по нему, вызывая метод `calculateArea()` для каждого элемента массива. Г)
Реализация инкапсуляции: объявите приватные поля в классе `Account` (`balance`), доступные только через публичные методы `deposit(amount)` и `withdraw(amount)`. создайте экземпляр класса `Account` и продемонстрируйте корректную работу методов.

Листинг

main.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include "class.hpp"

int main() {
    std::cout << "A:" << std::endl;
    Person p("Ivan", 25);
    p.displayInfo();

    std::cout << "\nB:" << std::endl;
    Student s("Anna", 20, 12345);
    s.displayInfo();

    std::cout << "\nC:" << std::endl;
    std::vector<Shape*> shapes;
    shapes.push_back(new Rectangle(5, 4));
    shapes.push_back(new Circle(3));

    for (Shape* s : shapes) {
        std::cout << "Area: " << s->calculateArea() << std::endl;
    }

    std::cout << "\nD:" << std::endl;
    Account acc(100.0);
    acc.deposit(50.0);
    acc.withdraw(30.0);
    std::cout << "Final balance: " << acc.getBalance() << std::endl;

    for (Shape* s : shapes) delete s;

    return 0;
}
```

class.hpp

```

#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int age;

public:
    Person(std::string n, int a) : name(n), age(a) {}

    void setName(std::string n) { name = n; }
    std::string getName() { return name; }

    virtual void displayInfo() {
        std::cout << "Name: " << name << ", Age: " << age << std::endl;
    }
    virtual ~Person() {}
};

class Student : public Person {
private:
    int studentId;

public:
    Student(std::string n, int a, int id) : Person(n, a), studentId(id) {}

    int getStudentId() { return studentId; }

    void displayInfo() override {
        std::cout << "Student: " << name << ", Age: " << age
            << ", ID: " << studentId << std::endl;
    }
};

class Shape {
public:
    virtual double calculateArea() = 0;
    virtual ~Shape() {}
};

class Rectangle : public Shape {
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double calculateArea() override { return width * height; }
};

class Circle : public Shape {

```

```
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    double calculateArea() override { return 3.14 * radius * radius; }
};

class Account {
private:
    double balance;

public:
    Account(double initialBalance) : balance(initialBalance) {}

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    bool withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    double getBalance() { return balance; }
};
```

Вывод

Я в очередной раз повторил работу с классами.